

Reviewer Pack — Minimal Local Cohomology of Ringed Spaces and Resolution Pro...

Entry 22771bd5ae8d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 13:12:32 UTC

Minimal Local Cohomology of Ringed Spaces and Resolution Proof Width Inequality

Entry ID: 22771bd5ae8d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 13:12:32 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Cohomology) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every Boolean formula φ with n variables, the resolution proof width $w(\varphi)$ is lower-bounded by a linear multiple of the minimal local cohomology degree $h(\varphi)$, such that $w(\varphi) = \Omega(h(\varphi)^2 \log n)$.

Rationale (proposer's reasoning):

Local cohomology provides a measure of complexity in algebraic geometry, potentially offering insights into computational complexity like resolution proof width. The conjecture suggests that higher local cohomology degrees correspond to more complex Boolean formulas.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c73a5fb905b22d49

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean formula φ with n variables ($n \leq 40$), we consider the resolution proof width $w(\varphi)$ and minimal local cohomology degree $h(\varphi)$. The conjecture is supported if the mean of the correlation coefficient between $h(\varphi)$ and $w(\varphi)$ across multiple instances exceeds 0.5, AND if the slope of the least squares fit line for this correlation is greater than or equal to $1.2 * \log n / h(\varphi)^2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Cohomology AND Resolution Proof Width IN Algebraic Geometry OR Boolean Satisfiability- $h(\phi)^2 \log n$ lower bound IN resolution proof complexity AND local cohomology degree IN algebraic geometry- Boolean formula ϕ resolution width $w(\phi)$ MINIMAL local cohomology $h(\phi)$ relationship

Top relevant hits considered: - [http://arxiv.org/abs/0707.3937v1] Cohomology theories for homotopy algebras and noncommutative geometry - [http://arxiv.org/abs/1209.3595v2] Noncommutative complex differential geometry - [http://arxiv.org/abs/1610.05741v2] Gerstenhaber bracket on the Hochschild cohomology via an arbitrary resolution - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_S^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/2604.05712v1] Precise measurement of the CKM angle γ with a novel approach - [http://arxiv.org/abs/2410.02421v2] Search for lepton number violating decays of $D_s^+ \rightarrow h^- h^0 e^+ e^+$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=0, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for _ in range(n):
            clause = random.sample(variables + [f'~{v}' for v in variables], n)
            clauses.append(' & '.join(clause))
        return ' | '.join(clauses)

    def local_cohomology_degree(formula, n):
        # Simplified version of local cohomology degree calculation
        # This is a placeholder and should be replaced with actual computation
        return random.randint(1, n)

    def resolution_proof_width(formula, n):
        # Simplified version of resolution proof width calculation
        # This is a placeholder and should be replaced with actual computation
        return random.randint(n, 2*n)
```

```

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    formula = generate_formula(n)
    h_phi = local_cohomology_degree(formula, n)
    w_phi = resolution_proof_width(formula, n)
    results.append((h_phi, w_phi))

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

h_values = [h for h, _ in results]
w_values = [w for _, w in results]
mean_h = sum(h_values) / len(h_values)
mean_w = sum(w_values) / len(w_values)
correlation_coefficient = sum((h - mean_h) * (w - mean_w) for h, w in results) / (len(results))
slope = correlation_coefficient * len(h_values) / sum((h - mean_h)**2 for h in h_values)

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for _, n in results),
    "conjecture_holds": correlation_coefficient > 0.5 and slope >= 1.2 * math.log(max(n for _, n in results)),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not all(result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")
    else:
        mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
        std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ample': ''}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.02255163529892894, 'instances_t
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.024630608638568356, 'instances_
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.023690629671912655, 'instances_
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.020225499713279416, 'instances_
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.018417012537494906, 'instances_
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.028138057915701065, 'instances_
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.016778317435075896, 'instances_
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.01935790024275251, 'instances_t
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.022176544551105636, 'instances_
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.021531850514935467, 'instances_
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': 0.021171911390068243, 'instances_
RESULT: FALSIFIED counterexample="first failing seed" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses random values for both local cohomology degree and resolution proof width, which are not meaningful measures of the actual properties they represent. The metric calculation is based on these random values, making it impossible to draw any conclusions about the conjecture's validity.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one instance (first failing seed=11), and the critic challenges the validity | next: Re-evaluate the conjecture with a more rigorous experimental design, ensuring meaningful measures are used for both local cohomology degree and resolution proof width.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22477
2	preregistration	ollama_remote	glm4:latest	0	0	11256
3	novelty	ollama_remote	glm4:latest	0	0	8528
4	novelty	ollama_remote	glm4:latest	0	0	22237
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22558
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	4589
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19613
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11822
9	critic	ollama_remote	glm4:latest	0	0	61472
10	judge	ollama_remote	glm4:latest	0	0	10119

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 194670 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/22771bd5ae8d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/22771bd5ae8d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/22771bd5ae8d.tar.gz` (if generated)